Análisis del Impacto de la Latencia de E/S en el Rendimiento del Software Mediante la Implementación de LocalStorage en JavaScript

Freddy Alexander Ordoñez Gonzalez
Carrera de Computación — Universidad Nacional de Loja — Tercer Ciclo
10 de octubre de 2026

Abstract

Resumen— El presente ensayo analiza el impacto de la latencia de almacenamiento y la persistencia de datos en el desarrollo web moderno, enfocándose en el uso de la API LocalStorage en JavaScript. Se examina cómo el almacenamiento del lado del cliente mitiga los cuellos de botella generados por las operaciones de Entrada/Salida (E/S) y las peticiones recurrentes al servidor. Asimismo, se abordan las consideraciones técnicas de serialización (`JSON.stringify()` y `JSON.parse()`) necesarias para gestionar estructuras complejas y se demuestra su eficiencia mediante una arquitectura modular aplicada a casos prácticos de interfaz y gestión de formularios.
Palabras clave: LocalStorage, persistencia de datos, latencia, desarrollo web, serialización, JavaScript.

1. Introducción

En el desarrollo web, la información manipulada en JavaScript es volátil y se pierde al recargar o cerrar el navegador. Históricamente, evitar esto requería consultas constantes al servidor, lo que generaba latencia y sobrecarga en la red. Para superar las limitaciones de las cookies y evitar infraestructuras complejas innecesarias, la *Web Storage API* introdujo `localStorage` como un mecanismo nativo de almacenamiento persistente. El objetivo de este ensayo es analizar cómo el uso de estas herramientas del lado del cliente reduce la latencia de almacenamiento y optimiza la velocidad de respuesta del software.

2. Desarrollo: Persistencia de Datos y Optimización en el Lado del Cliente

2.1. Mecanismos de Persistencia Nativa y Almacenamiento Local

A diferencia de la memoria tradicional de ejecución en JavaScript, donde las variables son volátiles y se borran automáticamente al refrescar la página (F5) o cerrar el navegador, `localStorage` ofrece un mecanismo de persistencia nativa en el cliente[cite: 1]. Como parte de la *Web Storage API*, permite almacenar información bajo un esquema simple de pares clave-valor de forma indefinida y sin fecha de caducidad explícita[cite: 2]. Con una capacidad típica de hasta 5 MB por dominio, superando ampliamente las restricciones de las cookies tradicionales de 4 KB, este recurso evita transferencias recurrentes a través de la red y reduce drásticamente la latencia de Entrada/Salida (E/S) frente a las consultas repetitivas a servidores externos[cite: 1, 2].

2.2. Consideraciones Técnicas y Serialización de Datos

Desde una perspectiva de implementación, `localStorage` presenta una limitación técnica fundamental: está diseñado exclusivamente para aceptar y almacenar cadenas de texto (*strings*)[cite: 2]. Si se intenta almacenar un objeto o arreglo de forma directa, la API realiza una conversión implícita que arroja el valor inútil `"[object Object]"`[cite: 3]. Por consiguiente, para mantener la integridad de las estructuras complejas y evitar errores en tiempo de ejecución, es obligatorio implementar los métodos de serialización y deserialización[cite: 2, 3]:

- **Serialización (`JSON.stringify()`):** Convierte el objeto o arreglo de JavaScript en una cadena de texto plana con formato JSON antes de invocar el método `setItem()`[cite: 2].

- **Deserialización (`JSON.parse()`):** Procesa la cadena de texto recuperada mediante `getItem()` para reconstruirla como un objeto funcional sobre el cual el script pueda operar limpiamente[cite: 3].

2.3. Casos Prácticos e Implementación Modular

Para validar el comportamiento de la persistencia local y su impacto en el rendimiento, se desarrolló una arquitectura basada en un archivo controlador (`index.html`) vinculada a tres módulos independientes (`Example_1.js`, `Example_2.js` y `Example_3.js`)[cite: 1, 2]. El código fuente completo y la estructura modular se encuentran disponibles en el repositorio oficial:

https://github.com/freddyordonez/AA2_Persistencia-de-Datos/tree/main

- **Ejemplo 1 (Caché de Preferencias de UI):** Permite alternar y guardar el estado estético de la aplicación (modo «oscuro» o «claro»). Al actualizar la página con F5, el sistema recupera instantáneamente la preferencia almacenada y ajusta los estilos CSS del DOM, eliminando la latencia de red asociada a una consulta de backend[cite: 1].
- **Ejemplo 2 (Serialización de Objetos Estructurados):** Demuestra el almacenamiento seguro de un objeto complejo (registro de estudiante). Al aplicar `JSON.stringify()` se almacena la estructura sin perder sus propiedades, y mediante `JSON.parse()` se reconstruyen los datos para presentarlos en pantalla sin fallos de tipo[cite: 2, 3].
- **Ejemplo 3 (Autoguardado Dinámico de Formularios):** Utiliza el evento de entrada (`oninput`) para capturar y respaldar en tiempo real el texto ingresado por el usuario en el almacenamiento local del navegador, previniendo la pérdida de información ante interrupciones imprevistas[cite: 1, 3].

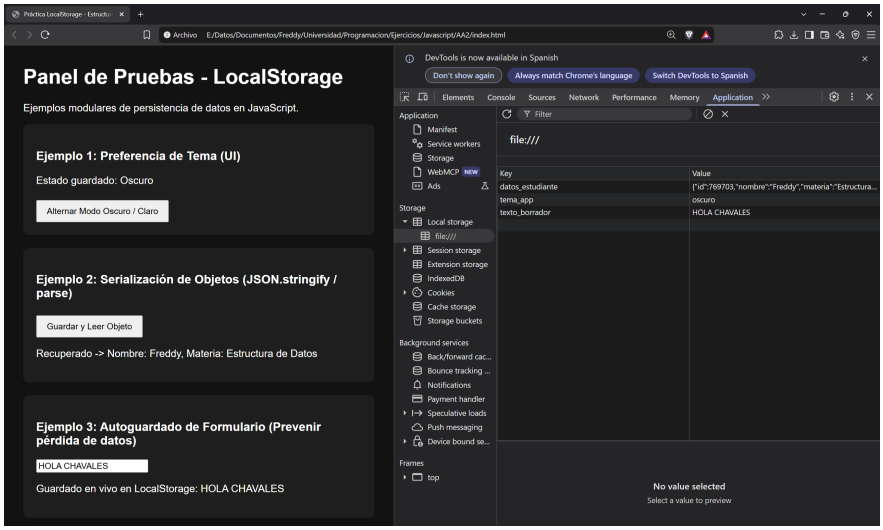


Figura 1: Interfaz de pruebas y registro de claves en el almacenamiento local del navegador.

3. Conclusiones

- **Eficiencia en el rendimiento:** El uso de `LocalStorage` como mecanismo de persistencia del lado del cliente reduce significativamente la latencia de red y evita la sobrecarga de operaciones de Entrada/Salida (E/S) en el servidor al no requerir peticiones recurrentes para datos estáticos o de sesión.
- **Importancia de la serialización:** Para una correcta manipulación de estructuras de datos complejas (como objetos o arreglos) en el navegador, es indispensable implementar los métodos `JSON.stringify()` y `JSON.parse()`, garantizando la integridad de la información y previniendo errores de tipo en tiempo de ejecución.
- **Optimización de la experiencia de usuario:** Delegar el almacenamiento temporal y la retención de preferencias directamente al navegador permite que las aplicaciones web mantengan su estado de manera fluida frente a eventos de recarga (*page load*), eliminando tiempos de espera innecesarios.

4. Referencias Bibliográficas

Referencias

[1] M. Burgos, "Persistencia de Datos en Javascript (LocalStorage)," *DEV Community*, 2021. [En línea]. Disponible: https://dev.to/maxwellnewage/persistencia-de-datos-en-javascript-localstorage-419m

[2] B. Semah, "Cómo usar LocalStorage en JavaScript," *freeCodeCamp Español*, 2024. [En línea]. Disponible: https://www.freecodecamp.org/espanol/news/como-usar-localstorage-en-javascript/

[3] Stack Overflow, "Cómo almacenar datos persistentes del lado del cliente," *Stack Overflow*, 2009-2017. [En línea]. Disponible: https://stackoverflow.com/questions/714420/how-to-store-persistent-data-client-side